TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements defined in the FRD into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It provides detailed technical design, data schemas, transformation logic, and runtime configurations necessary for code generation and deployment.

Scope

This TRD covers technical implementation details for:

- CSV file ingestion from S3 using AWS Glue DynamicFrame and Spark DataFrame APIs
- Data quality operations including null removal and deduplication using Spark transformations
- AWS Glue Data Catalog table registration and metadata management
- Apache Hudi integration for SCD Type 2 implementation with upsert operations
- Incremental processing logic using change data capture patterns
- Spark SQL aggregations for analytics table generation
- S3 write operations with appropriate file formats and partitioning strategies
- Error handling, logging, and operational monitoring patterns

This TRD does NOT cover:

- AWS CloudFormation or Terraform infrastructure code
- IAM role and policy definitions
- AWS Glue job scheduling or workflow orchestration configuration
- Cost optimization strategies
- Network configuration or VPC settings
- Backup and disaster recovery procedures

Assumptions

1. AWS Glue version 4.0 or higher with Spark 3.3+ and Python 3.10+ runtime
2. Apache Hudi 0.12.0 or higher libraries available in Glue job environment
3. CSV files use comma delimiter, UTF-8 encoding, and include header row

4. CustId data type: String (inferred from CSV)

5. OrderId data type: String (inferred from CSV)

6. PricePerUnit data type: Decimal(10,2) (inferred from business context)

7. Qty data type: Integer (inferred from business context)

8. Date field format: YYYY-MM-DD (ISO 8601 standard assumed)

9. TotalAmount calculation: Decimal(12,2) = PricePerUnit × Qty

10. OpTs (Operation Timestamp): Timestamp data type, generated using current_timestamp() function

11. IsActive: Boolean data type (true/false)

12. StartDate and EndDate: Date data type (YYYY-MM-DD)

13. Hudi table type: COPY_ON_WRITE for ordersummary table

14. Hudi record key for ordersummary: Composite key derived from OrderId + CustId

15. Hudi precombine field: OpTs (for conflict resolution)

16. "Null" string matching is case-sensitive

17. Duplicate detection considers all columns in the record

18. First occurrence of duplicate is retained (based on DataFrame row order)

19. Incremental processing baseline stored in S3 state location: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/

20. Glue job has read/write permissions to all specified S3 paths

21. Glue Data Catalog database gen_ai_poc_databrickscoe exists or job has permission to create it

22. Analytics table uses Parquet format with Snappy compression

23. No partitioning applied unless data volume exceeds 1GB per table

24. Athena query optimization not required in initial implementation

25. CloudWatch Logs retention period: 30 days (default)

•

2. Technical Requirements

TR-INGEST-001: Implement Customer Data Ingestion from S3

• Technical Requirement ID: TR-INGEST-001

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Implement S3 CSV file reader using AWS Glue DynamicFrame API to load customer data into Spark DataFrame for in-memory processing.

• Source Details:

• Source Type: S3 CSV Files

• Source Location: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV with header

• Source System: AWS S3

• Read Pattern: Batch read of all files in prefix

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: customer_raw_df

• Persistence: Transient (not persisted at this stage)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession

2. Use GlueContext.create_dynamic_frame.from_options() with format="csv"

3. Specify format_options: {"withHeader": "true", "separator": ","}

4. Convert DynamicFrame to Spark DataFrame using toDF()

5. Cache DataFrame in memory for subsequent operations

6. Log record count and schema information

• Validation Rules:

• Verify source path exists before read operation

• Validate that at least one CSV file is present in source location

• Confirm header row contains expected column names: CustId, Name, EmailId, Region

• Check that DataFrame is not empty after read operation

• Error Handling and Logging:

• If source path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/customerdata/ not found" and raise FileNotFoundError

• If no CSV files found: Log error "No CSV files found in source location" and raise ValueError

• If CSV parsing fails: Log error with file name and exception details, skip file, continue with remaining files

• If header validation fails: Log error "CSV header mismatch. Expected: CustId,Name,EmailId,Region" and raise ValueError

• If DataFrame is empty after read: Log warning "Customer data source is empty" but continue processing

• Log INFO level message: "Successfully loaded {record_count} customer records from S3"

• Runtime Parameters:

• source_path: s3://adif-sdlc/sdlc_wizard/customerdata/ (configurable via Glue job parameter --SOURCE_CUSTOMER_PATH)

• csv_separator: "," (default, configurable via --CSV_SEPARATOR)

• csv_encoding: "UTF-8" (default, configurable via --CSV_ENCODING)

• Operational Notes:

• Use Glue job bookmark to track processed files if incremental file processing is required in future

• For large file volumes (>1000 files), consider using S3 inventory for file discovery optimization

• Monitor S3 request throttling in CloudWatch metrics

•

TR-INGEST-002: Implement Order Data Ingestion from S3

• Technical Requirement ID: TR-INGEST-002

• Related Functional Requirement ID(s): FR-INGEST-002

• Objective:

Implement S3 CSV file reader using AWS Glue DynamicFrame API to load order transaction data into Spark DataFrame for in-memory processing.

• Source Details:

• Source Type: S3 CSV Files

• Source Location: s3://adif-sdlc/sdlc_wizard/orderdata/

• Source Format: CSV with header

• Source System: AWS S3

• Read Pattern: Batch read of all files in prefix

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: order_raw_df

• Persistence: Transient (not persisted at this stage)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	String	10 (YYYY-MM-DD)	Yes	
	CustId	String	Variable	No	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession (reuse from TR-INGEST-001 if in same job)

2. Use GlueContext.create_dynamic_frame.from_options() with format="csv"

3. Specify format_options: {"withHeader": "true", "separator": ","}

4. Convert DynamicFrame to Spark DataFrame using toDF()

5. Cache DataFrame in memory for subsequent operations

6. Log record count and schema information

• Validation Rules:

• Verify source path exists before read operation

• Validate that at least one CSV file is present in source location

• Confirm header row contains expected column names: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Check that DataFrame is not empty after read operation

• Validate Date column values match YYYY-MM-DD format pattern

• Error Handling and Logging:

• If source path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/orderdata/ not found" and raise FileNotFoundError

• If no CSV files found: Log error "No CSV files found in source location" and raise ValueError

• If CSV parsing fails: Log error with file name and exception details, skip file, continue with remaining files

• If header validation fails: Log error "CSV header mismatch. Expected: OrderId,ItemName,PricePerUnit,Qty,Date,CustId" and raise ValueError

• If DataFrame is empty after read: Log warning "Order data source is empty" but continue processing

• If Date format validation fails: Log warning "Invalid date format detected in {count} records" but continue processing (will be handled in cleaning stage)

• Log INFO level message: "Successfully loaded {record_count} order records from S3"

• Runtime Parameters:

• source_path: s3://adif-sdlc/sdlc_wizard/orderdata/ (configurable via Glue job parameter --SOURCE_ORDER_PATH)

• csv_separator: "," (default, configurable via --CSV_SEPARATOR)

• csv_encoding: "UTF-8" (default, configurable via --CSV_ENCODING)

• Operational Notes:

• Use Glue job bookmark to track processed files if incremental file processing is required in future

• For large file volumes (>1000 files), consider using S3 inventory for file discovery optimization

• Monitor S3 request throttling in CloudWatch metrics

• PricePerUnit and Qty columns may require explicit type casting during cleaning phase

•

TR-CLEAN-001: Implement Data Quality Processing for Customer Dataset

• Technical Requirement ID: TR-CLEAN-001

• Related Functional Requirement ID(s): FR-CLEAN-001

• Objective:

Implement Spark DataFrame transformations to remove null values (both string "Null" and actual NULL) and duplicate records from customer dataset.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: customer_raw_df (output from TR-INGEST-001)

• Source Location: Memory (transient)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: customer_clean_df

• Persistence: Transient (not persisted at this stage)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	

	----------	----------	----------	----------	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

• Output Schema:

Same as Input Schema (structure unchanged, content filtered)

• Processing / Transformation Logic:

1. Create filter condition to remove rows where any column equals string "Null" (case-sensitive):

• Use Spark SQL expression: `NOT (CustId = 'Null' OR Name = 'Null' OR EmailId = 'Null' OR Region = 'Null')`

2. Apply filter to remove string "Null" values

3. Apply dropna() transformation to remove rows with actual NULL values in any column

4. Apply dropDuplicates() transformation without specifying subset (considers all columns)

5. Log record counts before and after each cleaning operation

6. Persist cleaned DataFrame in memory with MEMORY_AND_DISK storage level

• Validation Rules:

• Verify that cleaned DataFrame contains no "Null" string values in any column

• Verify that cleaned DataFrame contains no NULL values in any column

• Verify that no duplicate rows exist in cleaned DataFrame

• Log warning if more than 50% of records are removed during cleaning

• Error Handling and Logging:

• If all records are removed: Log warning "All customer records removed during cleaning. Source data may be entirely invalid." Continue processing with empty DataFrame

• If no records are removed: Log info "No invalid or duplicate customer records found"

• Log INFO level message: "Customer cleaning complete. Input: {input_count}, Output: {output_count}, Removed: {removed_count}"

• Log detailed breakdown: "Removed {null_string_count} records with 'Null' strings, {null_value_count} records with NULL values, {duplicate_count} duplicate records"

• Runtime Parameters:

• null_string_value: "Null" (configurable via --NULL_STRING_VALUE)

• storage_level: "MEMORY_AND_DISK" (configurable via --STORAGE_LEVEL)

• Operational Notes:

• Consider using broadcast join optimization if customer dataset is small (<100MB)

• Monitor memory usage in Spark UI for large datasets

• If cleaning removes >80% of records, consider data quality investigation before proceeding

•

TR-CLEAN-002: Implement Data Quality Processing for Order Dataset

• Technical Requirement ID: TR-CLEAN-002

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Implement Spark DataFrame transformations to remove null values (both string "Null" and actual NULL) and duplicate records from order dataset.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: order_raw_df (output from TR-INGEST-002)

• Source Location: Memory (transient)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: order_clean_df

• Persistence: Transient (not persisted at this stage)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	String	10 (YYYY-MM-DD)	Yes	
	CustId	String	Variable	No	

• Output Schema:

Same as Input Schema (structure unchanged, content filtered)

• Processing / Transformation Logic:

1. Create filter condition to remove rows where any column equals string "Null" (case-sensitive):

• Use Spark SQL expression: `NOT (OrderId = 'Null' OR ItemName = 'Null' OR PricePerUnit = 'Null' OR Qty = 'Null' OR Date = 'Null' OR CustId = 'Null')`

2. Apply filter to remove string "Null" values

3. Apply dropna() transformation to remove rows with actual NULL values in any column

4. Apply dropDuplicates() transformation without specifying subset (considers all columns)

5. Cast PricePerUnit to DecimalType(10,2) if not already typed

6. Cast Qty to IntegerType if not already typed

7. Validate Date column format using regexp pattern matching (YYYY-MM-DD)

8. Log record counts before and after each cleaning operation

9. Persist cleaned DataFrame in memory with MEMORY_AND_DISK storage level

• Validation Rules:

• Verify that cleaned DataFrame contains no "Null" string values in any column

• Verify that cleaned DataFrame contains no NULL values in any column

• Verify that no duplicate rows exist in cleaned DataFrame

• Verify PricePerUnit values are positive decimals

• Verify Qty values are positive integers

• Verify Date values match YYYY-MM-DD format

• Log warning if more than 50% of records are removed during cleaning

• Error Handling and Logging:

• If all records are removed: Log warning "All order records removed during cleaning. Source data may be entirely invalid." Continue processing with empty DataFrame

• If no records are removed: Log info "No invalid or duplicate order records found"

• If type casting fails for PricePerUnit or Qty: Log error with record details and remove invalid records

• If Date format validation fails: Log warning with count of invalid dates and remove invalid records

• Log INFO level message: "Order cleaning complete. Input: {input_count}, Output: {output_count}, Removed: {removed_count}"

• Log detailed breakdown: "Removed {null_string_count} records with 'Null' strings, {null_value_count} records with NULL values, {duplicate_count} duplicate records, {invalid_type_count} records with invalid data types, {invalid_date_count} records with invalid dates"

• Runtime Parameters:

• null_string_value: "Null" (configurable via --NULL_STRING_VALUE)

• storage_level: "MEMORY_AND_DISK" (configurable via --STORAGE_LEVEL)

• date_format: "yyyy-MM-dd" (configurable via --DATE_FORMAT)

• Operational Notes:

• Monitor memory usage in Spark UI for large datasets

• If cleaning removes >80% of records, consider data quality investigation before proceeding

• Consider implementing data quality metrics dashboard in CloudWatch for ongoing monitoring

•

TR-CATALOG-001: Implement Glue Data Catalog Registration for Cleaned Datasets

• Technical Requirement ID: TR-CATALOG-001

• Related Functional Requirement ID(s): FR-CATALOG-001

• Objective:

Register cleaned customer and order DataFrames as tables in AWS Glue Data Catalog using Glue DynamicFrame write operations with catalog integration.

• Source Details:

• Source Type: In-memory Spark DataFrames

• Source Names: customer_clean_df (from TR-CLEAN-001), order_clean_df (from TR-CLEAN-002)

• Source Location: Memory (transient)

• Target Details:

• Target Type: AWS Glue Data Catalog Tables

• Target Database: gen_ai_poc_databrickscoe

• Target Tables: sdlc_wizard_customer, sdlc_wizard_order

• Storage Location (Customer): s3://adif-sdlc/curated/sdlc_wizard/customer/

• Storage Location (Order): s3://adif-sdlc/curated/sdlc_wizard/order/

• Storage Format: Parquet with Snappy compression

• Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	

	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema:

Same as Input Schemas (direct mapping to catalog tables)

• Processing / Transformation Logic:

1. Check if database "gen_ai_poc_databrickscoe" exists in Glue Data Catalog using boto3 glue client

2. If database does not exist, create database using create_database() API

3. Convert customer_clean_df to DynamicFrame using DynamicFrame.fromDF()

4. Write customer DynamicFrame to S3 using write_dynamic_frame.from_catalog() with:

• database: gen_ai_poc_databrickscoe

• table_name: sdlc_wizard_customer

• transformation_ctx: "customer_catalog_write"

• format: "parquet"

• compression: "snappy"

• mode: "overwrite"

5. Convert order_clean_df to DynamicFrame using DynamicFrame.fromDF()

6. Cast Date column from String to Date type before writing

7. Write order DynamicFrame to S3 using write_dynamic_frame.from_catalog() with:

• database: gen_ai_poc_databrickscoe

• table_name: sdlc_wizard_order

• transformation_ctx: "order_catalog_write"

• format: "parquet"

• compression: "snappy"

• mode: "overwrite"

8. Verify table registration by querying Glue Data Catalog using get_table() API

9. Log table metadata including record count, schema, and S3 location

• Validation Rules:

• Verify database exists or is successfully created

• Verify both tables are successfully registered in catalog

• Verify S3 locations contain Parquet files after write operations

• Verify table schemas in catalog match expected schemas

• Verify tables are queryable via Athena (optional validation query)

• Error Handling and Logging:

• If database creation fails: Log error "Failed to create database gen_ai_poc_databrickscoe: {error_message}" and raise exception

• If table write fails: Log error "Failed to write table {table_name}: {error_message}" and raise exception

• If S3 write permission denied: Log error "S3 write permission denied for location {s3_path}" and raise exception

• If catalog registration fails: Log error "Failed to register table {table_name} in catalog: {error_message}" and raise exception

• Log INFO level message: "Successfully registered table {table_name} with {record_count} records at {s3_location}"

• Log table schema details at DEBUG level

• Runtime Parameters:

• catalog_database: gen_ai_poc_databrickscoe (configurable via --CATALOG_DATABASE)

• customer_table_name: sdlc_wizard_customer (configurable via --CUSTOMER_TABLE_NAME)

• order_table_name: sdlc_wizard_order (configurable via --ORDER_TABLE_NAME)

• customer_s3_location: s3://adif-sdlc/curated/sdlc_wizard/customer/ (configurable via --CUSTOMER_S3_LOCATION)

• order_s3_location: s3://adif-sdlc/curated/sdlc_wizard/order/ (configurable via --ORDER_S3_LOCATION)

• file_format: parquet (configurable via --FILE_FORMAT)

• compression: snappy (configurable via --COMPRESSION)

• write_mode: overwrite (configurable via --WRITE_MODE)

• Operational Notes:

• Use overwrite mode for initial load; consider append mode for incremental updates in future iterations

• Monitor S3 storage costs as Parquet files accumulate

• Consider implementing S3 lifecycle policies for data retention

• Verify Athena query performance after catalog registration

• Use Glue Data Catalog versioning to track schema evolution

•

TR-SCD2-001: Implement SCD Type 2 Logic for Order Summary Using Apache Hudi

• Technical Requirement ID: TR-SCD2-001

• Related Functional Requirement ID(s): FR-SCD2-001

• Objective:

Implement Slowly Changing Dimension Type 2 logic using Apache Hudi COPY_ON_WRITE table format to maintain historical versions of order summary records with proper versioning and active record tracking.

• Source Details:

• Source Type: AWS Glue Data Catalog Tables

• Source Tables:

• gen_ai_poc_databrickscoe.sdlc_wizard_customer (from TR-CATALOG-001)

• gen_ai_poc_databrickscoe.sdlc_wizard_order (from TR-CATALOG-001)

• Source Location:

• s3://adif-sdlc/curated/sdlc_wizard/customer/

• s3://adif-sdlc/curated/sdlc_wizard/order/

• Target Details:

• Target Type: Apache Hudi Table (COPY_ON_WRITE)

• Target Table: ordersummary

• Target Database: gen_ai_poc_databrickscoe

• Target Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Hudi Table Type: COPY_ON_WRITE

• Hudi Operation: UPSERT

• Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	

	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema (Order Summary):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	OrderId	String	Variable	No	Direct map from Order.OrderId	
	CustId	String	Variable	No	Direct map from Order.CustId	
	Name	String	Variable	No	Direct map from Customer.Name	
	EmailId	String	Variable	No	Direct map from Customer.EmailId	
	Region	String	Variable	No	Direct map from Customer.Region	
	ItemName	String	Variable	No	Direct map from Order.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from Order.PricePerUnit	
	Qty	Integer	-	No	Direct map from Order.Qty	
	Date	Date	-	No	Direct map from Order.Date	
	IsActive	Boolean	-	No	Derived: true for current version, false for historical	
	StartDate	Date	-	No	Derived: current_date() for new/updated records	
	EndDate	Date	-	Yes	Derived: null for active records, current_date() for closed records	
	OpTs	Timestamp	-	No	Derived: current_timestamp()	

• Processing / Transformation Logic:

• Step 1: Read Source Data

1. Read customer table from Glue Catalog: `spark.read.table("gen_ai_poc_databrickscoe.sdlc_wizard_customer")`

2. Read order table from Glue Catalog: `spark.read.table("gen_ai_poc_databrickscoe.sdlc_wizard_order")`

• Step 2: Join Customer and Order Data

1. Perform inner join on CustId: `order_df.join(customer_df, "CustId", "inner")`

2. Select all columns from both tables

3. Add SCD2 metadata columns:

• IsActive = lit(True)

• StartDate = current_date()

• EndDate = lit(None).cast(DateType())

• OpTs = current_timestamp()

• Step 3: Read Existing Hudi Table (if exists)

1. Check if Hudi table exists at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

2. If exists, read using: `spark.read.format("hudi").load(hudi_table_path)`

3. If not exists, create empty DataFrame with ordersummary schema

• Step 4: Implement SCD2 Logic

1. Define business key: Composite key of OrderId + CustId

2. Identify new records: Records in joined data not present in existing Hudi table (based on business key)

3. Identify changed records: Records in joined data with same business key but different attribute values compared to active records in Hudi table

4. Identify unchanged records: Records with same business key and identical attribute values

• For New Records:

• Insert as-is with IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

• For Changed Records:

• Close existing active record: Update IsActive=false, EndDate=current_date()

• Insert new active record: IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

• For Unchanged Records:

• No action required

• Step 5: Write to Hudi Table

1. Configure Hudi write options:

```
hudi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'OrderId,CustId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.partitionpath.field': '',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.hive_style_partitioning': 'false',
'hoodie.upsert.shuffle.parallelism': 2,
'hoodie.insert.shuffle.parallelism': 2
}
```

2. Write DataFrame using: `df.write.format("hudi").options(hudi_options).mode("append").save(hudi_table_path)`

• Step 6: Register/Update Glue Catalog

1. Use Hudi's Glue sync utility or manual catalog registration

2. Update table metadata in gen_ai_poc_databrickscoe.ordersummary

• Validation Rules:

• Verify that only one active record (IsActive=true) exists per business key (OrderId + CustId)

• Verify that all historical records have IsActive=false and non-null EndDate

• Verify that all active records have IsActive=true and null EndDate

• Verify that StartDate <= EndDate for all closed records

• Verify that OpTs is populated for all records

• Verify join produces expected record count (no unexpected cartesian products)

• Error Handling and Logging:

• If join produces zero records: Log warning "Join between customer and order tables produced no results" and create empty Hudi table

• If Hudi write fails: Log error "Hudi upsert operation failed: {error_message}" and raise exception

• If multiple active records detected for same business key: Log error "Data integrity violation: Multiple active records found for business key {key}" and raise exception

• If Hudi table path is not accessible: Log error "Cannot access Hudi table location {path}" and raise exception

• Log INFO level message: "SCD2 processing complete. New records: {new_count}, Changed records: {changed_count}, Unchanged records: {unchanged_count}"

• Log Hudi commit metadata at DEBUG level

• Runtime Parameters:

• hudi_table_path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (configurable via --HUDI_TABLE_PATH)

• hudi_table_name: ordersummary (configurable via --HUDI_TABLE_NAME)

• hudi_record_key: OrderId,CustId (configurable via --HUDI_RECORD_KEY)

• hudi_precombine_field: OpTs (configurable via --HUDI_PRECOMBINE_FIELD)

• hudi_table_type: COPY_ON_WRITE (configurable via --HUDI_TABLE_TYPE)

• hudi_operation: upsert (configurable via --HUDI_OPERATION)

• catalog_database: gen_ai_poc_databrickscoe (configurable via --CATALOG_DATABASE)

• catalog_table: ordersummary (configurable via --CATALOG_TABLE)

• Operational Notes:

• Hudi COPY_ON_WRITE provides snapshot isolation and better read performance

• Monitor Hudi commit timeline for compaction and cleaning operations

• Consider implementing Hudi clustering for query performance optimization

• Use Hudi's incremental query capability for downstream consumers

• Monitor S3 storage growth due to Hudi's versioning mechanism

• Implement Hudi table services (cleaning, compaction) in separate maintenance job

•

TR-INCR-001: Implement Incremental Processing Logic Based on Customer Changes

• Technical Requirement ID: TR-INCR-001

• Related Functional Requirement ID(s): FR-INCR-001

• Objective:

Implement change data capture (CDC) pattern to detect customer data changes and trigger corresponding SCD2 updates in ordersummary table, optimizing processing by handling only changed records.

• Source Details:

• Current Customer Data Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (from TR-INGEST-001)

• Baseline Customer Data Source: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/ (previous execution state)

• Order Summary Hudi Table: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (from TR-SCD2-001)

• Target Details:

• Target Type: Apache Hudi Table (COPY_ON_WRITE)

• Target Table: ordersummary (updated in-place)

• Target Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Baseline Update Location: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/

• Input Schema (Current Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Input Schema (Baseline Customer):

Same as Current Customer (or empty on first run)

• Output Schema:

Same as TR-SCD2-001 ordersummary schema (updates applied to existing table)

• Processing / Transformation Logic:

• Step 1: Load Current and Baseline Customer Data

1. Load current customer data from s3://adif-sdlc/sdlc_wizard/customerdata/ (cleaned data from TR-CLEAN-001)

2. Check if baseline exists at s3://adif-sdlc/state/sdlc_wizard/customer_baseline/

3. If baseline exists, load baseline customer data

4. If baseline does not exist (first run), treat all current records as new

• Step 2: Detect Customer Changes

1. Perform full outer join between current and baseline on CustId

2. Identify change types:

• New Customers: CustId present in current but not in baseline

• Modified Customers: CustId present in both, but Name, EmailId, or Region values differ

• Unchanged Customers: CustId present in both with identical attribute values

• Deleted Customers: CustId present in baseline but not in current (optional handling)

3. Create changed_customers_df containing only new and modified customer records

• Step 3: Filter Affected Order Summary Records

1. Read active records from ordersummary Hudi table: `filter(col("IsActive") == True)`

2. Join changed_customers_df with active ordersummary records on CustId

3. Identify all order summary records that need SCD2 update due to customer changes

• Step 4: Apply SCD2 Updates

1. For each affected order summary record:

• Close existing active record: Set IsActive=false, EndDate=current_date()

• Create new active record with updated customer attributes: IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

2. Prepare upsert DataFrame with both closed and new records

3. Write to Hudi table using upsert operation (same configuration as TR-SCD2-001)

• Step 5: Update Baseline

1. Write current customer data to baseline location: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/

2. Use overwrite mode to replace previous baseline

3. Log baseline update timestamp

• Validation Rules:

• Verify that change detection logic correctly identifies new, modified, and unchanged records

• Verify that only affected order summary records are updated (not full table refresh)

• Verify that baseline is successfully updated after processing

• Verify that no active records are orphaned (all active records have valid customer references)

• Error Handling and Logging:

• If baseline does not exist on first run: Log info "Baseline not found. Treating all records as new." and proceed with full load

• If change detection produces zero changes: Log info "No customer changes detected. Skipping SCD2 updates." and exit gracefully

• If baseline update fails: Log error "Failed to update customer baseline: {error_message}" but do not fail job (baseline will be stale for next run)

• If Hudi upsert fails: Log error "Incremental SCD2 update failed: {error_message}" and raise exception

• Log INFO level message: "Incremental processing complete. New customers: {new_count}, Modified customers: {modified_count}, Affected order summary records: {affected_count}"

• Log change detection metrics at INFO level

• Runtime Parameters:

• current_customer_path: s3://adif-sdlc/sdlc_wizard/customerdata/ (configurable via --CURRENT_CUSTOMER_PATH)

• baseline_customer_path: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/ (configurable via --BASELINE_CUSTOMER_PATH)

• ordersummary_hudi_path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (configurable via --ORDERSUMMARY_HUDI_PATH)

• enable_incremental: true (configurable via --ENABLE_INCREMENTAL, set to false for full refresh)

• Operational Notes:

• First execution will perform full load since baseline does not exist

• Subsequent executions will process only changed customer records

• Consider implementing soft delete handling for deleted customers (currently not specified in FRD)

• Monitor baseline storage size and implement retention policy if needed

• Use Spark DataFrame caching for change detection operations to optimize performance

• Consider implementing checkpointing for long-running incremental jobs

•

TR-AGG-001: Implement Customer Aggregate Spend Analytics Table

• Technical Requirement ID: TR-AGG-001

• Related Functional Requirement ID(s): FR-AGG-001

• Objective:

Implement Spark SQL aggregation logic to calculate total customer spending by customer name and date, producing an analytics table for business intelligence consumption.

• Source Details:

• Source Type: Apache Hudi Table

• Source Table: gen_ai_poc_databrickscoe.ordersummary

• Source Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:

• Target Type: Parquet Table

• Target Table: customeraggregatespend

• Target Database: gen_ai_poc_databrickscoe

• Target Location: s3://adif-sdlc/analytics/customeraggregatespend/

• Target Format: Parquet with Snappy compression

• Write Mode: Overwrite

• Input Schema (Order Summary - Active Records Only):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	IsActive	Boolean	-	No	
	StartDate	Date	-	No	
	EndDate	Date	-	Yes	
	OpTs	Timestamp	-	No	

• Output Schema (Customer Aggregate Spend):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	Name	String	Variable	No	Direct map from ordersummary.Name	
	TotalAmount	Decimal	12,2	No	Derived: SUM(PricePerUnit × Qty) grouped by Name, Date	

	Date	Date	-	No	Direct map from ordersummary.Date	

• Processing / Transformation Logic:

• Step 1: Read Active Order Summary Records

1. Read ordersummary Hudi table from Glue Catalog: `spark.read.table("gen_ai_poc_databrickscoe.ordersummary")`

2. Filter for active records only: `filter(col("IsActive") == True)`

3. Select required columns: Name, PricePerUnit, Qty, Date

• Step 2: Calculate Total Amount

1. Create calculated column TotalAmount: `col("PricePerUnit") col("Qty")`

2. Cast result to DecimalType(12,2) to ensure precision

• Step 3: Aggregate by Customer and Date

1. Group by Name and Date

2. Apply SUM aggregation on TotalAmount: `groupBy("Name", "Date").agg(sum("TotalAmount").alias("TotalAmount"))`

3. Select final columns in order: Name, TotalAmount, Date

• Step 4: Write to Analytics Location

1. Write DataFrame to S3 using Parquet format:

```
df.write
.format("parquet")
.option("compression", "snappy")
.mode("overwrite")
.save("s3://adif-sdlc/analytics/customeraggregatespend/")
```

• Step 5: Register in Glue Catalog

1. Create or update table customeraggregatespend in gen_ai_poc_databrickscoe database

2. Point table location to s3://adif-sdlc/analytics/customeraggregatespend/

3. Register schema metadata

• Validation Rules:

• Verify that TotalAmount is never negative

• Verify that TotalAmount is never NULL (should be 0 if no orders)

• Verify that aggregation produces expected number of unique Name-Date combinations

• Verify that all Name values are non-null

• Verify that all Date values are valid dates

• Error Handling and Logging:

• If ordersummary table is empty: Log warning "Order summary table is empty. Creating empty customeraggregatespend table." and create empty table with schema

• If no active records found: Log warning "No active records in order summary. Creating empty customeraggregatespend table." and create empty table

• If TotalAmount calculation produces NULL: Log warning "NULL TotalAmount detected for Name={name}, Date={date}" and replace with 0

• If aggregation produces negative TotalAmount: Log error "Negative TotalAmount detected: {amount} for Name={name}, Date={date}" and raise data quality exception

• If write to analytics location fails: Log error "Failed to write customeraggregatespend table: {error_message}" and raise exception

• Log INFO level message: "Customer aggregate spend calculation complete. Total customers: {customer_count}, Total date ranges: {date_count}, Total records: {record_count}"

• Log aggregation statistics at INFO level: "Total spend across all customers: {total_spend}"

• Runtime Parameters:

• source_table: gen_ai_poc_databrickscoe.ordersummary (configurable via --SOURCE_TABLE)

• target_path: s3://adif-sdlc/analytics/customeraggregatespend/ (configurable via --TARGET_PATH)

• target_database: gen_ai_poc_databrickscoe (configurable via --TARGET_DATABASE)

• target_table: customeraggregatespend (configurable via --TARGET_TABLE)

• file_format: parquet (configurable via --FILE_FORMAT)

• compression: snappy (configurable via --COMPRESSION)

• write_mode: overwrite (configurable via --WRITE_MODE)

• Operational Notes:

• Use overwrite mode to ensure analytics table reflects current active state

• Consider implementing partitioning by Date if data volume grows significantly (>1GB)

• Monitor query performance in Athena after table creation

• Consider implementing incremental aggregation if full refresh becomes performance bottleneck

• Implement data quality checks to validate aggregation results

• Consider adding additional aggregation dimensions (Region, ItemName) in future iterations

•

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python 3.10+
- Framework: PySpark 3.3+ (AWS Glue 4.0 runtime)
- Additional Libraries:
- Apache Hudi 0.12.0+ (for SCD2 implementation)
- boto3 (for AWS service interactions)
- awsglue (AWS Glue Python library)

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Execution Mode: Batch processing
- Parallelism: Configurable via Glue DPU allocation (recommended: 2-10 DPUs based on data volume)
- Execution Frequency: Scheduled (daily recommended for incremental processing)
- Job Timeout: 2880 minutes (48 hours maximum, adjust based on data volume)
- Max Retries: 3 (configurable)
- Job Bookmark: Enabled for incremental file processing (optional)

Glue Job Type Expectations

- Glue Version: 4.0 (Spark 3.3.0, Python 3.10)
- Worker Type: G.1X or G.2X (based on data volume)
- Number of Workers: 2-10 (auto-scaling recommended)
- Max Capacity: Not applicable (using worker type configuration)
- Connections: None required (S3 access via IAM role)
- Security Configuration: Optional (for encryption at rest and in transit)
- Script Location: s3:///scripts/glue_etl_pipeline.py
- Temporary Directory: s3:///temp/
- Spark UI Logs: s3:///sparkui-logs/
- CloudWatch Logs: Enabled with log group /aws-glue/jobs/output and /aws-glue/jobs/error
-

4. Data Design Details

Source Paths / Source Systems

- Customer CSV Files: s3://adif-sdlc/sdlc_wizard/customerdata/
- Format: CSV with header
- Encoding: UTF-8
- Delimiter: Comma

• Source System: External data feed (assumed)

• Order CSV Files: s3://adif-sdlc/sdlc_wizard/orderdata/
• Format: CSV with header
• Encoding: UTF-8
• Delimiter: Comma
• Source System: External data feed (assumed)

• Customer Baseline State: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/
• Format: Parquet
• Purpose: Incremental processing change detection
• Created by: TR-INCR-001

Target Paths / Target Tables

• Curated Layer:
• Customer Table:
• Path: s3://adif-sdlc/curated/sdlc_wizard/customer/
• Format: Parquet with Snappy compression
• Catalog: gen_ai_poc_databrickscoe.sdlc_wizard_customer
• Update Behavior: Overwrite on each run

• Order Table:
• Path: s3://adif-sdlc/curated/sdlc_wizard/order/
• Format: Parquet with Snappy compression
• Catalog: gen_ai_poc_databrickscoe.sdlc_wizard_order
• Update Behavior: Overwrite on each run

• Order Summary Hudi Table:
• Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
• Format: Apache Hudi (COPY_ON_WRITE)
• Catalog: gen_ai_poc_databrickscoe.ordersummary
• Update Behavior: Upsert (SCD2 logic)
• Record Key: OrderId, CustId (composite)
• Precombine Field: OpTs

• Analytics Layer:
• Customer Aggregate Spend Table:
• Path: s3://adif-sdlc/analytics/customeraggregatespend/

- Format: Parquet with Snappy compression
- Catalog: gen_ai_poc_databrickscoe.customeraggregatespend
- Update Behavior: Overwrite on each run

Catalog Registration Expectations

- Database: gen_ai_poc_databrickscoe
- Create if not exists
- Location: Not specified (metadata only)

- Tables:
- sdlc_wizard_customer: Registered via Glue DynamicFrame write
- sdlc_wizard_order: Registered via Glue DynamicFrame write
- ordersummary: Registered via Hudi Glue sync or manual registration
- customeraggregatespend: Registered via Glue DynamicFrame write or manual registration

- Schema Evolution: Not enabled (strict schema enforcement)
- Table Properties: Default Glue table properties
- Athena Compatibility: All tables must be queryable via Athena

Partitioning Expectations

- Customer Table: No partitioning (small dataset expected)
- Order Table: No partitioning initially (consider partitioning by Date if volume exceeds 1GB)
- Order Summary Hudi Table: No partitioning initially (Hudi manages internal partitioning)
- Customer Aggregate Spend Table: No partitioning initially (consider partitioning by Date if volume exceeds 1GB)

- Future Partitioning Considerations:
- If order data grows beyond 1GB, implement partitioning by Date (YYYY-MM-DD)
- If customer aggregate spend grows beyond 1GB, implement partitioning by Date
- Hudi table may benefit from custom partitioning strategy based on query patterns

File Formats / Table Formats

- CSV: Source files only (customer and order data)
- Parquet: Curated and analytics tables (customer, order, customeraggregatespend)
- Compression: Snappy (balance between compression ratio and query performance)
- Row Group Size: Default (128MB)
- Page Size: Default (1MB)

- Apache Hudi: Order summary table (SCD2 implementation)

• Table Type: COPY_ON_WRITE (optimized for read-heavy workloads)

• Payload Class: Default (OverwriteWithLatestAvroPayload)

• Index Type: BLOOM (default for COPY_ON_WRITE)

• Compaction Strategy: Inline compaction disabled (run separately)

• Cleaning Strategy: Async cleaning (retain 10 commits)

Update / Overwrite / Append Behavior

• Customer Table (sdlc_wizard_customer):

• Write Mode: Overwrite

• Rationale: Full refresh on each run to ensure data consistency

• Behavior: Existing data is deleted and replaced with current cleaned data

• Order Table (sdlc_wizard_order):

• Write Mode: Overwrite

• Rationale: Full refresh on each run to ensure data consistency

• Behavior: Existing data is deleted and replaced with current cleaned data

• Order Summary Hudi Table (ordersummary):

• Write Mode: Append (with Upsert operation)

• Rationale: SCD2 requires historical preservation and incremental updates

• Behavior:

• New records are inserted

• Changed records trigger close of old version and insert of new version

• Unchanged records are not modified

• Hudi manages versioning and active record tracking

• Customer Aggregate Spend Table (customeraggregatespend):

• Write Mode: Overwrite

• Rationale: Analytics table reflects current active state of order summary

• Behavior: Existing data is deleted and replaced with freshly aggregated data from active order summary records

• Customer Baseline State:

• Write Mode: Overwrite

• Rationale: Baseline represents snapshot of customer data from previous run

• Behavior: Previous baseline is replaced with current customer data after successful processing

Data Retention and Lifecycle

• Source CSV Files: Retention managed externally (not specified in FRD)

• Curated Tables: No automatic retention policy (manual cleanup required)

• Hudi Table: Retain 10 commits (configurable via Hudi cleaning policy)

• Analytics Tables: No automatic retention policy (manual cleanup required)

• Baseline State: Retain only latest snapshot (overwrite on each run)

• Recommendation: Implement S3 lifecycle policies for cost optimization:

• Archive curated data older than 90 days to S3 Glacier

• Delete temporary files older than 7 days

• Retain Hudi commit metadata for audit trail

•

5. Processing Details

Data Quality Rules

• Null Handling:

• String "Null" Values: Remove any record where any column contains the exact string "Null" (case-sensitive)

• Actual NULL Values: Remove any record where any column contains actual NULL value

• Implementation: Apply filter condition using Spark SQL NOT operator combined with OR conditions for all columns

• Scope: Applied to both customer and order datasets during cleaning phase (TR-CLEAN-001, TR-CLEAN-002)

• Data Type Validation:

• PricePerUnit: Must be positive decimal value, cast to DecimalType(10,2)

• Qty: Must be positive integer value, cast to IntegerType

• Date: Must match YYYY-MM-DD format, cast to DateType

• Invalid Records: Remove records that fail type casting or validation

• Implementation: Use Spark withColumn() with cast() and filter() operations

• Business Rule Validation:

• CustId: Must be non-null and non-empty string (primary key for customer)

• OrderId: Must be non-null and non-empty string (primary key for order)

• TotalAmount: Must be non-negative after calculation (PricePerUnit × Qty >= 0)

• IsActive: Only one active record (IsActive=true) per business key in ordersummary table

• Date Ranges: StartDate <= EndDate for all closed records in ordersummary table

Deduplication Rules

- Customer Deduplication:
- Scope: All columns (CustId, Name, EmailId, Region)
- Method: Spark dropDuplicates() without subset parameter
- Retention: First occurrence of duplicate record is retained
- Order: Based on DataFrame row order (non-deterministic unless explicitly sorted)
- Implementation: `customer_df.dropDuplicates()`

- Order Deduplication:
- Scope: All columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)
- Method: Spark dropDuplicates() without subset parameter
- Retention: First occurrence of duplicate record is retained
- Order: Based on DataFrame row order (non-deterministic unless explicitly sorted)
- Implementation: `order_df.dropDuplicates()`

- SCD2 Deduplication:
- Scope: Business key (OrderId + CustId) with IsActive=true
- Method: Hudi upsert operation with precombine field (OpTs)
- Retention: Latest record based on OpTs timestamp
- Validation: Post-processing check to ensure only one active record per business key

Join Rules

- Customer-Order Join (TR-SCD2-001):
- Join Type: Inner join
- Join Key: CustId
- Join Condition: `order_df.join(customer_df, "CustId", "inner")`
- Cardinality: One-to-many (one customer to many orders)
- Expected Behavior: Each order record is enriched with corresponding customer attributes
- Null Handling: Inner join automatically excludes records with NULL CustId
- Duplicate Handling: If customer data contains duplicates (should not after cleaning), join may produce cartesian product
- Optimization: Broadcast join if customer dataset is small (<10MB)

- Change Detection Join (TR-INCR-001):
- Join Type: Full outer join
- Join Key: CustId
- Join Condition: `current_customer_df.join(baseline_customer_df, "CustId", "outer")`

• Purpose: Identify new, modified, and deleted customer records

• Change Detection Logic:

• New: CustId in current but not in baseline (baseline columns are NULL)

• Modified: CustId in both, but Name/EmailId/Region differ

• Unchanged: CustId in both with identical attributes

• Deleted: CustId in baseline but not in current (current columns are NULL)

• Order Summary Filter Join (TR-INCR-001):

• Join Type: Inner join

• Join Key: CustId

• Join Condition: `changed_customers_df.join(ordersummary_active_df, "CustId", "inner")`

• Purpose: Identify order summary records affected by customer changes

• Filter: Only active records (IsActive=true) from ordersummary

Aggregation Rules

• Customer Aggregate Spend (TR-AGG-001):

• Aggregation Function: SUM

• Aggregation Expression: `SUM(PricePerUnit × Qty)`

• Group By Columns: Name, Date

• Calculated Column: TotalAmount = PricePerUnit × Qty (calculated before aggregation)

• Data Type: DecimalType(12,2) for TotalAmount

• Null Handling: If PricePerUnit or Qty is NULL (should not occur after cleaning), result is NULL; replace with 0

• Implementation:

```python
df.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
.groupBy("Name", "Date")
.agg(sum("TotalAmount").alias("TotalAmount"))
.select("Name", "TotalAmount", "Date")
```

• Aggregation Validation:

• Verify TotalAmount is never negative

• Verify TotalAmount is never NULL (replace with 0 if needed)

• Verify aggregation produces expected number of unique Name-Date combinations

SCD / History Tracking Rules

• SCD Type 2 Implementation (TR-SCD2-001):

• Business Key: Composite key of OrderId + CustId (uniquely identifies an order-customer relationship)

• Versioning Columns:

• IsActive (Boolean):

• true = current active version

• false = historical closed version

• Only one active record per business key at any time

• StartDate (Date):

• Date when this version became active

• Set to current_date() when record is inserted or updated

• EndDate (Date):

• Date when this version was closed (became inactive)

• NULL for active records

• Set to current_date() when record is closed due to change

• OpTs (Timestamp):

• Operation timestamp indicating when record was processed

• Set to current_timestamp() for all insert/update operations

• Used as Hudi precombine field for conflict resolution

• Change Detection Logic:

• New Record: Business key not present in existing ordersummary table

• Action: Insert with IsActive=true, StartDate=current_date(), EndDate=NULL, OpTs=current_timestamp()

• Changed Record: Business key exists but attribute values differ

• Action 1: Close existing active record (set IsActive=false, EndDate=current_date())

• Action 2: Insert new active record with updated values (IsActive=true, StartDate=current_date(), EndDate=NULL, OpTs=current_timestamp())

• Unchanged Record: Business key exists with identical attribute values

• Action: No modification (existing active record remains unchanged)

• Hudi Configuration for SCD2:

• Table Type: COPY_ON_WRITE (optimized for read performance)

• Operation: UPSERT (insert new records, update existing records)

• Record Key: OrderId,CustId (composite business key)

• Precombine Field: OpTs (latest timestamp wins in case of conflicts)

• Partition Path: Empty (no partitioning initially)

• Payload Class: OverwriteWithLatestAvroPayload (default)

• Historical Query Patterns:

• Current State: `SELECT FROM ordersummary WHERE IsActive = true`

• Historical State at Date: `SELECT FROM ordersummary WHERE StartDate <= '2024-01-01' AND (EndDate IS NULL OR EndDate > '2024-01-01')`

• Change History: `SELECT FROM ordersummary WHERE OrderId = 'O123' AND CustId = 'C456' ORDER BY StartDate`

Null Handling

• Source Data Null Handling:

• String "Null": Treated as invalid data, records removed during cleaning

• Actual NULL: Treated as missing data, records removed during cleaning

• Scope: All columns in customer and order datasets

• Implementation: Two-stage filter:

1. Remove string "Null" values using filter condition

2. Remove actual NULL values using dropna()

• Derived Column Null Handling:

• TotalAmount Calculation: If PricePerUnit or Qty is NULL (should not occur after cleaning), result is NULL

• Mitigation: Replace NULL with 0 using coalesce() or fillna()

• SCD2 Metadata Columns:

• IsActive: Never NULL (always true or false)

• StartDate: Never NULL (always set to current_date())

• EndDate: NULL for active records, non-NULL for closed records

• OpTs: Never NULL (always set to current_timestamp())

• Join Null Handling:

• Inner Join: Automatically excludes records with NULL join keys

• Outer Join: Preserves records with NULL join keys (used for change detection)

• Null Propagation: Columns from non-matching side of outer join are NULL

• Aggregation Null Handling:

• SUM Function: Ignores NULL values in aggregation (treats as 0)

• COUNT Function: Counts non-NULL values only

• Recommendation: Use coalesce() to replace NULL with 0 before aggregation for clarity

Type Casting Expectations

• Customer Data Type Casting:

• CustId: String (no casting required, inferred from CSV)

• Name: String (no casting required, inferred from CSV)

• EmailId: String (no casting required, inferred from CSV)

• Region: String (no casting required, inferred from CSV)

• Order Data Type Casting:

• OrderId: String (no casting required, inferred from CSV)

• ItemName: String (no casting required, inferred from CSV)

• PricePerUnit: Cast from String to DecimalType(10,2)

• Implementation: `col("PricePerUnit").cast(DecimalType(10,2))`

• Validation: Remove records where casting fails or result is negative

• Qty: Cast from String to IntegerType

• Implementation: `col("Qty").cast(IntegerType())`

• Validation: Remove records where casting fails or result is negative

• Date: Cast from String to DateType

• Implementation: `to_date(col("Date"), "yyyy-MM-dd")`

• Validation: Remove records where casting fails (invalid date format)

• CustId: String (no casting required, inferred from CSV)

• Derived Column Type Casting:

• TotalAmount: DecimalType(12,2)

• Calculation: `(col("PricePerUnit") col("Qty")).cast(DecimalType(12,2))`

• Rationale: Accommodate larger values from multiplication

• IsActive: BooleanType

• Values: true or false

• Implementation: `lit(True).cast(BooleanType())`

• StartDate: DateType

• Implementation: `current_date()`

• EndDate: DateType (nullable)

• Implementation: `lit(None).cast(DateType())` for active records

• OpTs: TimestampType

• Implementation: `current_timestamp()`

• Type Casting Error Handling:

• If casting fails, record is removed and logged

• Log error message includes column name, original value, and target type

• Count of failed casts is logged at INFO level

• If >50% of records fail casting, raise data quality exception

•

6. Traceability Matrix

	Technical Source
	----------
	s3://adif-sdlc/sdlc_wizard/customerdata/
	s3://adif-sdlc/sdlc_wizard/orderdata/
1)	customer_raw_df (from TR-INGEST-001)
	order_raw_df (from TR-INGEST-002)
	customer_clean_df, order_clean_df
customer, gen_ai_poc_databrickscoe.sdlc_wizard_order	gen_ai_poc_databrickscoe.sdlc_wizard_customer, gen_ai_poc_databrickscoe.sdlc_wizard
, s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	s3://adif-sdlc/sdlc_wizard/customerdata/, s3://adif-sdlc/state/sdlc_wizard/customer_baseli
y	gen_ai_poc_databrickscoe.ordersummary

• Source Fidelity Verification:

• All TRD technical sources match the original sources specified in the FRD

• TR-INCR-001 introduces baseline state location (s3://adif-sdlc/state/sdlc_wizard/customer_baselin e/) which is a technical implementation detail for change detection, not a replacement for the original source

• No intermediate staging tables or alternative sources have been introduced

• All catalog table references maintain the original database and table names from the FRD

• Dependency Chain:

1. TR-INGEST-001, TR-INGEST-002 (parallel, no dependencies)

2. TR-CLEAN-001 (depends on TR-INGEST-001), TR-CLEAN-002 (depends on TR-INGEST-002)

3. TR-CATALOG-001 (depends on TR-CLEAN-001, TR-CLEAN-002)

4. TR-SCD2-001 (depends on TR-CATALOG-001)

5. TR-INCR-001 (depends on TR-SCD2-001, optional for subsequent runs)

6. TR-AGG-001 (depends on TR-SCD2-001)

•

7. ETL Mapping Requirements

TR-INGEST-001: Customer Data Ingestion Mapping

• Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	---	---	---	---	---	---	
	CustId	String	Variable	No	CustId	Direct map	
	Name	String	Variable	Yes	Name	Direct map	
	EmailId	String	Variable	Yes	EmailId	Direct map	
	Region	String	Variable	Yes	Region	Direct map	

• Target: customer_raw_df (In-memory DataFrame)

•

TR-INGEST-002: Order Data Ingestion Mapping

• Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	---	---	---	---	---	---	
	OrderId	String	Variable	No	OrderId	Direct map	
	ItemName	String	Variable	Yes	ItemName	Direct map	
	PricePerUnit	String	Variable	Yes	PricePerUnit	Direct map (cast to Decimal in TR-CLEAN-002)	
	Qty	String	Variable	Yes	Qty	Direct map (cast to Integer in TR-CLEAN-002)	
	Date	String	10	Yes	Date	Direct map (cast to Date in TR-CLEAN-002)	
	CustId	String	Variable	No	CustId	Direct map	

• Target: order_raw_df (In-memory DataFrame)

•

TR-CLEAN-001: Customer Data Cleaning Mapping

• Source: customer_raw_df (In-memory DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	---	---	---	---	---	---	
	CustId	String	Variable	No	CustId	Filter: Remove if = "Null" or NULL	

	Name	String	Variable	Yes	Name	Filter: Remove if = "Null" or NULL	
	EmailId	String	Variable	Yes	EmailId	Filter: Remove if = "Null" or NULL	
	Region	String	Variable	Yes	Region	Filter: Remove if = "Null" or NULL	

• Target: customer_clean_df (In-memory DataFrame)

• Additional Transformations:

• Remove duplicate records (all columns considered)

• Retain first occurrence of duplicates

•

TR-CLEAN-002: Order Data Cleaning Mapping

• Source: order_raw_df (In-memory DataFrame)

e	Length/Precision/Scale	Nullable	Target Column	Target Data Type	Transformation
	----------	----------	----------	----------	----------
	Variable	No	OrderId	String	Filter: Remove if = "Null" or NULL
	Variable	Yes	ItemName	String	Filter: Remove if = "Null" or NULL
	Variable	Yes	PricePerUnit	Decimal(10,2)	Filter: Remove if = "Null" or NULL; Cast to Decimal(10,2); Remov
	Variable	Yes	Qty	Integer	Filter: Remove if = "Null" or NULL; Cast to Integer; Remove if neg
	10	Yes	Date	Date	Filter: Remove if = "Null" or NULL; Cast to Date using format yyyy
	Variable	No	CustId	String	Filter: Remove if = "Null" or NULL

• Target: order_clean_df (In-memory DataFrame)

• Additional Transformations:

• Remove duplicate records (all columns considered)

• Retain first occurrence of duplicates

•

TR-CATALOG-001: Glue Catalog Registration Mapping

• Source 1: customer_clean_df → Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	----------	----------	----------	----------	----------	----------	
	CustId	String	Variable	No	CustId	Direct map	
	Name	String	Variable	No	Name	Direct map	
	EmailId	String	Variable	No	EmailId	Direct map	
	Region	String	Variable	No	Region	Direct map	

• Storage: s3://adif-sdlc/curated/sdlc_wizard/customer/ (Parquet, Snappy)

• Source 2: order_clean_df → Target: gen_ai_poc_databrickscoe.sdlc_wizard_order

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	---	---	---	---	---	---	
	OrderId	String	Variable	No	OrderId	Direct map	
	ItemName	String	Variable	No	ItemName	Direct map	
	PricePerUnit	Decimal	10,2	No	PricePerUnit	Direct map	
	Qty	Integer	-	No	Qty	Direct map	
	Date	Date	-	No	Date	Direct map	
	CustId	String	Variable	No	CustId	Direct map	

• Storage: s3://adif-sdlc/curated/sdlc_wizard/order/ (Parquet, Snappy)

•

TR-SCD2-001: Order Summary SCD Type 2 Mapping

• Source 1: gen_ai_poc_databrickscoe.sdlc_wizard_customer

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Source 2: gen_ai_poc_databrickscoe.sdlc_wizard_order

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Target: gen_ai_poc_databrickscoe.ordersummary (Hudi COPY_ON_WRITE)

Column	Data Type	Length/Precision/Scale	Nullable	Source Mapping	Transformation
---	---	---	---	---	---

d	String	Variable	No	order.OrderId	Direct map from order table
	String	Variable	No	order.CustId	Direct map from order table (join key)
	String	Variable	No	customer.Name	Direct map from customer table via join on CustId
d	String	Variable	No	customer.EmailId	Direct map from customer table via join on CustId
	String	Variable	No	customer.Region	Direct map from customer table via join on CustId
ame	String	Variable	No	order.ItemName	Direct map from order table
erUnit	Decimal	10,2	No	order.PricePerUnit	Direct map from order table
	Integer	-	No	order.Qty	Direct map from order table
	Date	-	No	order.Date	Direct map from order table
ve	Boolean	-	No	N/A	Derived: lit(True) for new/updated records
ate	Date	-	No	N/A	Derived: current_date()
te	Date	-	Yes	N/A	Derived: lit(None) for active records, current_date() for closed r
	Timestamp	-	No	N/A	Derived: current_timestamp()

• Storage: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (Hudi COPY_ON_WRITE)

• Join Logic:

• Inner join: order.CustId = customer.CustId

• Result: All order columns + all customer columns (excluding duplicate CustId)

• SCD2 Logic:

• New records: Insert with IsActive=true, StartDate=current_date(), EndDate=null

• Changed records: Close old (IsActive=false, EndDate=current_date()), Insert new (IsActive=true, StartDate=current_date(), EndDate=null)

• Unchanged records: No action

•

TR-INCR-001: Incremental Processing Mapping

• Source 1: s3://adif-sdlc/sdlc_wizard/customerdata/ (Current Customer Data)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Source 2: s3://adif-sdlc/state/sdlc_wizard/customer_baseline/ (Baseline Customer Data)

	Source Column	Data Type	Length/Precision/Scale	Nullable	

	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Change Detection Logic:

• Full outer join on CustId

• New: CustId in current but not in baseline

• Modified: CustId in both, but Name/EmailId/Region differ

• Unchanged: CustId in both with identical attributes

• Deleted: CustId in baseline but not in current (optional handling)

• Target: gen_ai_poc_databrickscoe.ordersummary (Hudi COPY_ON_WRITE) - Updated in place

• Affected Records: Order summary records where CustId matches changed customers

• SCD2 Update Logic:

• For each affected order summary record:

• Close existing active record: IsActive=false, EndDate=current_date()

• Insert new active record with updated customer attributes: IsActive=true, StartDate=current_date(), EndDate=null, OpTs=current_timestamp()

• Baseline Update:

• Write current customer data to s3://adif-sdlc/state/sdlc_wizard/customer_baseline/ (overwrite mode)

•

TR-AGG-001: Customer Aggregate Spend Mapping

• Source: gen_ai_poc_databrickscoe.ordersummary (Active Records Only)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Used In Aggregation	
	---	---	---	---	---	
	OrderId	String	Variable	No	No	
	CustId	String	Variable	No	No	
	Name	String	Variable	No	Yes (Group By)	
	EmailId	String	Variable	No	No	
	Region	String	Variable	No	No	
	ItemName	String	Variable	No	No	
	PricePerUnit	Decimal	10,2	No	Yes (Calculation)	

	Qty	Integer	-	No	Yes (Calculation)	
	Date	Date	-	No	Yes (Group By)	
	IsActive	Boolean	-	No	Yes (Filter: IsActive = true)	
	StartDate	Date	-	No	No	
	EndDate	Date	-	Yes	No	
	OpTs	Timestamp	-	No	No	

• Target: gen_ai_poc_databrickscoe.customeraggregatespend

Data Type	Length/Precision/Scale	Nullable	Source Mapping	Transformation
---	---	---	---	---
String	Variable	No	ordersummary.Name	Direct map (Group By key)
Decimal	12,2	No	ordersummary.PricePerUnit, ordersummary.Qty	Derived: SUM(PricePerUnit × Qty) groupe
Date	-	No	ordersummary.Date	Direct map (Group By key)

• Storage: s3://adif-sdlc/analytics/customeraggregatespend/ (Parquet, Snappy)

• Aggregation Logic:

1. Filter: IsActive = true
2. Calculate: TotalAmount = PricePerUnit × Qty (per record)
3. Group By: Name, Date
4. Aggregate: SUM(TotalAmount)
5. Select: Name, TotalAmount, Date

•

Additional Notes

Schema Details Not Available in FRD

The following schema details were not explicitly specified in the FRD and have been inferred based on common data engineering practices and business context:

1. Customer Table:

• CustId data type: Assumed String (common for customer identifiers)

• Name, EmailId, Region data types: Assumed String

• Column lengths: Not specified, using Variable length

2. Order Table:

• OrderId data type: Assumed String (common for order identifiers)

• ItemName data type: Assumed String

• PricePerUnit precision/scale: Assumed Decimal(10,2) (common for currency values)

• Qty data type: Assumed Integer (common for quantity fields)

• Date format: Assumed YYYY-MM-DD (ISO 8601 standard)

• Column lengths: Not specified, using Variable length for strings

3. Order Summary Table:

• TotalAmount precision/scale: Assumed Decimal(12,2) (larger than PricePerUnit to accommodate multiplication)

• IsActive data type: Assumed Boolean (standard for SCD2 active flags)

• StartDate, EndDate data types: Assumed Date (standard for SCD2 date ranges)

• OpTs data type: Assumed Timestamp (standard for operation timestamps)

4. Customer Aggregate Spend Table:

• TotalAmount precision/scale: Assumed Decimal(12,2) (consistent with order summary)

If more precise schema information becomes available, these assumptions should be updated accordingly.

•

• Document Version: 1.0

• Last Updated: 2025-01-10

• Status: Ready for Code Generation

• Prepared By: AWS Glue Solution Engineer

• Reviewed By: [Pending Review]